

Lulo

Security Assessment

December 30th, 2024 — Prepared by OtterSec

Tamta Topuria tamta@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **3**

Overview 3

Key Findings 3

Scope **4**

Findings **5**

Vulnerabilities **6**

OS-LUO-ADV-00 | Unconstrained Allocation-Based Withdrawals 9

OS-LUO-ADV-01 | Fund Drainage Due to Missing Pending Withdrawals 11

OS-LUO-ADV-02 | Failure to Segregate Referral Supply Across Allocations 12

OS-LUO-ADV-03 | Manipulation of Referrer Metrics 14

OS-LUO-ADV-04 | Repeated Subtraction of Claimable Amount 16

OS-LUO-ADV-05 | Static Nature of Configured Weights 18

OS-LUO-ADV-06 | Risk of Outdated Allocation Data 20

OS-LUO-ADV-07 | Inaccuracies in Exchange Rate Calculation 21

OS-LUO-ADV-08 | Validation Failure Due to Rounding Error 23

OS-LUO-ADV-09 | Improper Oracle Address Check 24

OS-LUO-ADV-10 | Utilization of Shared Pool PDA as Signer 25

OS-LUO-ADV-11 | Incorrect Oracle Staleness Check 26

OS-LUO-ADV-12 | Inconsistencies in Conversion of Protocol Identifiers 27

OS-LUO-ADV-13 | Transaction Failure Due to Drift Withdrawal Limit 29

OS-LUO-ADV-14 | Unfavorable Rounding Direction 30

General Findings **31**

© 2024 Otter Audits LLC. All Rights Reserved.

OS-LUO-SUG-00 | Lack of Price Freshness Validation 32

OS-LUO-SUG-01 | Token Account Fragmentation 33

OS-LUO-SUG-02 | Absence of Ownership Verification 34

OS-LUO-SUG-03 | Missing Validation Logic 35

OS-LUO-SUG-04 | Code Maturity 36

OS-LUO-SUG-05 | Code Refactoring 37

OS-LUO-SUG-06 | Redundant Code 39

Appendices

Vulnerability Rating Scale **40**

Procedure **41**

© 2024 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Lulo Labs engaged OtterSec to assess the `flexlend` program. This assessment was conducted between November 13th and December 23rd, 2024. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 22 findings throughout this audit engagement.

In particular, we identified several critical vulnerabilities, including one where users may withdraw all of their `PUSD` to bypass allocation checks and withdraw stablecoins they did not originally deposit, resulting in token shortages and potential arbitrage opportunities (OS-LUO-ADV-00), and the failure to segregate referral supply across allocations with different exchange rates, which allows referrers to exploit the system by claiming unearned rewards from allocations with more favorable rates (OS-LUO-ADV-02).

We also highlighted the possibility of exploiting Lulo's referral system by repeatedly recycling funds, falsely assigning a referrer, and withdrawing funds, thereby depleting the referrer's metrics without contributing to their rewards (OS-LUO-ADV-03). Furthermore, rounding down token amounts during withdrawals will result in discrepancies between token burns and allocations, exposing the system to small but cumulative financial losses (OS-LUO-ADV-14).

There is also a high-severity vulnerability concerning the protocol weights, which are set based on initial exposure limits that may exceed the updated maximum exposure due to changes in fund distribution, resulting in overexposure beyond the protocol's intended coverage capacity (OS-LUO-ADV-05). Additionally, there is another issue regarding the absence of a staleness check in the withdrawal process, which may result in outdated allocation data and create inaccuracies in exchange rate calculations and withdrawal validations (OS-LUO-ADV-06). Furthermore, protocol names exceeding 8 bytes are truncated, which may result in collisions between protocols with similar prefixes, and attempting to copy more than 8 bytes into the fixed-length identifier will result in a panic (OS-LUO-ADV-12).

We also recommended modifying the codebase for improved efficiency (OS-LUO-SUG-05) and made suggestions regarding inconsistencies in the codebase to ensure adherence to best development practices (OS-LUO-SUG-04). We further advised that the user account's program-derived address be controlled by the Lulo program (OS-LUO-SUG-02) and that proper validation be performed to mitigate potential security issues (OS-LUO-SUG-03).

© 2024 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/lulo-labs/flexlend. This audit was performed against commit fd28989.

A brief description of the program is as follows:

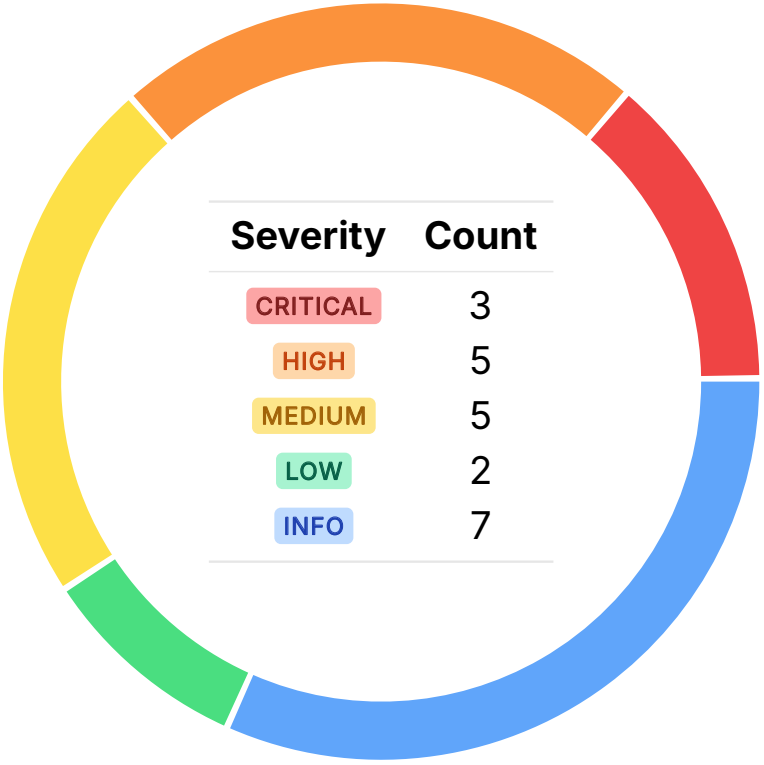
Name	Description
flexlend	The system consists of two pools: the Protected Pool (`PUSD`) for secure, low-yield investments, and the Regular Pool (`LUSD`) for high-yield, high-risk opportunities.

© 2024 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 22 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	3
HIGH	5
MEDIUM	5
LOW	2
INFO	7

© 2024 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-LUO-ADV-00	CRITICAL	RESOLVED	Users may withdraw all of their `PUSD` to bypass allocation checks and withdraw stablecoins they did not originally deposit, resulting in token shortages and potential arbitrage opportunities.
OS-LUO-ADV-01	CRITICAL	RESOLVED	The pool allows simultaneous withdrawals without accounting for pending withdrawals, resulting in potential over-commitment and depletion of `LUSD` funds from the pool.
OS-LUO-ADV-02	CRITICAL	RESOLVED	The protocol fails to segregate referral supply across allocations with varying exchange rates, allowing referrers to exploit the system by claiming unearned rewards from allocations offering more favorable rates.
OS-LUO-ADV-03	HIGH	RESOLVED	It is possible to exploit Lulo's referral system by repeatedly recycling funds, falsely assigning referrers, and withdrawing, thereby depleting the referrers' metrics without contributing to their rewards.
OS-LUO-ADV-04	HIGH	RESOLVED	`claimable_amount` is subtracted once via `unclaimed_referral_fees` and again from `total_referred_amount` resulting in inaccurate referral exchange rate calculations and unfairly reducing referrers' rewards.
OS-LUO-ADV-05	HIGH	RESOLVED	The protocol weights are set based on initial exposure limits, which may exceed the updated maximum exposure due to changes in fund distribution, resulting in overexposure beyond the protocol's intended coverage capacity.

© 2024 Otter Audits LLC. All Rights Reserved.

OS-LUO-ADV-06	HIGH	RESOLVED	The absence of `error_if_stale_refresh` in `initiate_regular_withdraw` may result in outdated allocation data, leading to inaccuracies in exchange rate calculations and withdrawal validations.
OS-LUO-ADV-07	HIGH	RESOLVED	The exchange rate is calculated via partially updated state variables, resulting in inaccuracies when determining referral collateral amounts and leading to unfair fund distributions.
OS-LUO-ADV-08	MEDIUM	RESOLVED	`protected_rewards` and `regular_rewards` in `calculate_accumulated_split` are rounded, resulting in a minor discrepancy when their sum is compared to `accumulated_net`.
OS-LUO-ADV-09	MEDIUM	RESOLVED	`update_oracle_price` improperly checks the `write_authority` of the provided oracle account instead of verifying its address against the `Allocation`'s oracle address.
OS-LUO-ADV-10	MEDIUM	RESOLVED	Utilizing a single pool PDA as the signer for CPI interactions across multiple protocols creates a shared authority, where a breach or unauthorized update in one protocol will compromise funds in others.
OS-LUO-ADV-11	MEDIUM	RESOLVED	The `elapsed_time` variable, which tracks the pool's last refresh, is incorrectly utilized to check the staleness of the oracle, instead of the correct `oracle_elapsed` variable, which tracks the time since the oracle was last updated.
OS-LUO-ADV-12	MEDIUM	RESOLVED	Protocol names exceeding 8 bytes are truncated, which may result in collisions between protocols with similar prefixes. Additionally, attempting to copy more than 8 bytes into the fixed-length identifier will result in a panic.
OS-LUO-ADV-13	LOW	RESOLVED	The Drift protocol's daily withdrawal limits may cause transactions to fail when attempting regular withdrawals, leading to user inconvenience, operational delays, and potential liquidity management challenges.

© 2024 Otter Audits LLC. All Rights Reserved.

OS-LUO-ADV-14	LOW	RESOLVED	Rounding down token amounts during withdrawals will result in discrepancies between token burns and allocations, exposing the system to small but cumulative financial losses.

© 2024 Otter Audits LLC. All Rights Reserved.

Unconstrained Allocation-Based Withdrawals CRITICAL OS-LUO-ADV-00

Description

The vulnerability in `withdraw_allocation` arises from insufficient validation when handling `withdraw_all` requests. Specifically, if the `withdraw_all` flag is set to true, users are permitted to withdraw more of an asset than they deposited.

>_ *flexlend/src/utils/math.rs* RUST

```rust
/// this check prevents a user from withdrawing an asset that was not deposited
pub fn withdraw_allocation(
    &mut self,
    allocation_index: usize,
    withdraw_supply: u64,
    withdraw_all: bool,
) -> Result<()> {
   [...]
    // TODO should still recheck this
    if withdraw_supply > deposited_supply {
        msg!(
            "user withdraw supply {} > deposited_supply {}, withdraw_all={}",
            withdraw_supply,
            deposited_supply,
            withdraw_all
        );
        require!(withdraw_all, Errors::InvalidPoolWithdraw);
    }
    self.deposited_allocations[allocation_index] = self.deposited_allocations[allocation_index]
            .checked_sub(withdraw_supply)
            .unwrap_or(0);
    [...]
}
```

The current logic in `withdraw_allocation` allows a withdrawal to proceed even if the `withdraw_supply` (the amount of stablecoin to be withdrawn) exceeds the `deposited_supply` (the amount the user deposited) of the user-requested token, provided the `withdraw_all` flag is set to true. Consequently, a single user may withdraw more of a stablecoin than they deposited, creating an imbalance between the deposited and available assets within the protocol.

© 2024 Otter Audits LLC. All Rights Reserved.

Proof of Concept

The above vulnerability is illustrated in the following scenario:

1. The user deposits `1 USDT`, resulting in an equivalent issuance of `PUSD` (a protocol stablecoin). They also deposit `500 USDC`, resulting in another equivalent issuance of `PUSD`.
2. The user invokes the `withdraw_protected_pool` instruction with `USDT` as `input_mint` and `501` as `amount`. Because the user is withdrawing all of their `PUSD`, the `withdraw_all` variable is set to `true`.
3. Since the requested `withdraw_supply` (`501`) exceeds the `deposited_supply` (`1`), the condition (`if withdraw_supply > deposited_supply`) is triggered. However, because `withdraw_all == true` , the withdrawal is permitted even though the amount exceeds their deposit.
4. The withdrawal logic effectively allows the user to take `501 USDT`. The remaining `500 USDT` comes from the protocol's `USDT` pool, which is backed by deposits made by other users.

Remediation

Ensure that the requested withdrawal amount aligns with the amount of a specific token that the user deposited.

Patch

Resolved in 31c973e and 986d24d.

© 2024 Otter Audits LLC. All Rights Reserved.

Fund Drainage Due to Missing Pending Withdrawals CRITICAL OS-LUO-ADV-01

Description

It is possible for the pool to be drained because of inconsistencies in how `pending_withdrawals` are handled during withdrawal validation and finalization in `initiate_regular_pool_withdraw` and `complete_regular_pool_withdraw`, respectively. `regular_withdraw_allowed` is only called in `initiate_regular_pool_withdraw`, which checks the remaining regular liquidity amount (`regular_liquidity_amount`) after accounting for the requested withdrawal (`amount`).

>_ *flexlend/src/utils/checks.rs* RUST

```rust
pub fn regular_withdraw_allowed(pool: &Pool, amount: u64) -> Result<bool> {
    let updated_regular = pool
        .regular_liquidity_amount()?
        .checked_sub(amount)
        .ok_or(error!(Errors::UnderflowError))?;
    [...]
}
```

However, it does not subtract the funds held in `pending_withdrawals` that have been approved but not yet deducted from the pool's balance. Since pending withdrawals are not deducted from the pool's regular liquidity at the time of validation, multiple users initiating withdrawals simultaneously may bypass the pool's capacity checks in `initiate_regular_pool_withdraw`. These pending withdrawals are finally subtracted from the regular amount in `complete_regular_pool_withdraw`. Consequently, as this instruction does not call `regular_withdraw_allowed`, if all `LUSD` holders decide to withdraw their tokens simultaneously, the pool will be drained of all `LUSD`.

This issue also impacts `PUSD` deposits, as `protected_deposit_allowed` fails to consider pending withdrawals of regular tokens. This oversight allows users to deposit into the `PUSD` pool even when a significant portion of `LUSD`s is withdrawn, potentially destabilizing the system.

Remediation

Ensure `regular_withdraw_allowed` subtracts `pending_withdrawals` from the total regular amount, or call `regular_withdraw_allowed` in the `complete_regular_pool_withdraw` instruction.

Patch

Resolved in b18e98c.

© 2024 Otter Audits LLC. All Rights Reserved.

Failure to Segregate Referral Supply Across Allocations CRITICAL OS-LUO-ADV-02

Description

The vulnerability lies in the centralized accounting of referral supply (`referral_supply`) in the `UserAccount` structure, without differentiating between allocations (`USDC` and `PYUSD`). Each allocation (stablecoin or asset) has its own referral exchange rate, derived from varying APYs or market dynamics in associated liquidity pools. The `UserAccount` accumulates the `referral_supply` from different allocations into a single variable.

>_ *flexlend/src/state/user.rs* RUST

```rust
pub fn calculate_claimable_referral_rewards(
    &self,
    pool_mint: &Allocation,
) -> Result<(u64, u64)> {
    let referral_exchange_rate = pool_mint.referral_exchange_rate()?;
    [...]
    let claimable_c_amount = referral_supply
        .checked_sub(referred_c_amount)
        .unwrap_or(Fraction::ZERO);
    let claimable_amount =
        referral_exchange_rate.fraction_collateral_to_liquidity(claimable_c_amount);
    [...]
}
```

Consequently, when a referrer receives referral supply from one allocation, instead of claiming rewards from the original asset, they may select a different asset's allocation that has a more favorable exchange rate. This enables the referrer to disproportionately inflate their rewards, sometimes withdrawing rewards that they have not earned.

Proof of Concept

1. Suppose `USDC`'s `referral_exchange_rate` is $15/17$, and `PYUSD`'s `referral_exchange_rate` is $15/19$.
2. A user deposits 100 `USDC`. Their `referrer` receives 100 in `referred_amount` and 88 in `referral_supply` ($100 * (15/17)$).
3. The `referrer` calls `claim_referral_fees` with `PYUSD` as `input_mint`, attempting to claim referral fees utilizing `PYUSD` instead of `USDC` (even though the rewards were generated from the `USDC` allocation).

© 2024 Otter Audits LLC. All Rights Reserved.

(a) `claimable_c_amount` = `referral_supply` - `referred_c_amount` = $88 - (100 * (15/19))$ = $88 - 78$ = 10.
(b) `claimable_amount` = `claimable_c_amount` / `referral_exchange_rate` = $10/(15/19)$ = 12 `PYUSD`.

4. As a result, the `referrer` is rewarded 12 `PYUSD`, despite no referrals originating from `PYUSD` deposits.

Remediation

Maintain separate `referral_supply` and `referral_amount` for each allocation within the `UserAccount` .

Patch

Resolved in 8166c5e.

© 2024 Otter Audits LLC. All Rights Reserved.

Manipulation of Referrer Metrics HIGH OS-LUO-ADV-03

Description

The vulnerability lies in how `update_referrer_on_withdraw` adjusts the `referred_amount` and `referral_supply` for a referrer, even when the withdrawal originates from funds that were not contributed to the referrer's referred amount. This flaw enables an attacker to prevent any referrer from receiving their rewards. When depositing into Lulo, having a referrer is not mandatory. Deposits made without a referrer may later be assigned to one during a subsequent deposit. The protocol associates any withdrawal with the referrer without verifying whether the withdrawn funds are directly attributable to their referred amount.

>_ flexlend/src/state/pool/liquidity.rs RUST

```rust
pub fn update_referrer_on_withdraw([...]) -> Result<()> {
    [...]
    if withdraw_amount >= referrer.referred_amount {
        referrer.referral_supply = 0;
        referrer.referred_amount = 0;
        self.total_referred_amount -= referrer.referred_amount;
        self.total_referral_supply -= referrer.referral_supply;
    }else {[...]
        referrer.referred_amount = referrer
            .referred_amount
            .checked_sub(withdraw_amount)
            .unwrap_or(0);
        referrer.referral_supply = referrer
            .referral_supply
            .checked_sub(claimed_c_amount)
            .unwrap_or(0);
        [...]
    }
    Ok(())
}
```

This vulnerability allows attackers to associate funds with a referrer that were never actually referred. When assets deposited without a referrer are withdrawn, the referrer's variables in `update_referrer_on_withdraw` are still reduced. This allows users to decrease a referrer's `referred_amount` without contributing to it, potentially preventing the referrer from claiming rewards by repeatedly recycling funds to drain the referrer's referred amount.

© 2024 Otter Audits LLC. All Rights Reserved.

Proof of Concept

1. Bob deposits 100,000 `USDC` and sets Alice as his `referrer`. As a result, Alice's `referred_amount` becomes 100,000 `USDC`. Alice's `referral_supply` and related rewards start accruing.
2. Eve (the attacker) deposits 10,000 `USDC` without specifying a `referrer`. This amount remains unlinked to any referral rewards or metrics.
3. Eve deposits an additional small amount, 1 `USDC`, and sets Alice as her `referrer` during this deposit. This manipulates the protocol to increase Alice's `referred_amount` to 100,001 `USDC` (Bob's deposit + Eve's small deposit).
4. Thereafter, Eve withdraws 10,000 `USDC`, which decreases Alice's `referred_amount` as well, despite the fact that Eve's 10,000 `USDC` was not part of Alice's `referred_amount`.
5. Eve repeats steps 2 through 4 with a new `UserAccount`, continuing this cycle until Alice's `referred_amount` is completely drained. Thus, Eve may deplete Alice's `referrer` rewards without needing a full `referred_amount` by repeatedly cycling the same 10,000 `USDC` until Alice's `referred_amount` is entirely drained.

Remediation

Allow setting a referrer only in the initial deposit.

Patch

Resolved in 7438d8c.

© 2024 Otter Audits LLC. All Rights Reserved.

Repeated Subtraction of Claimable Amount HIGH OS-LUO-ADV-04

Description

In the withdrawal instructions, `claimable_amount` is deducted twice in the exchange rate calculation. First, it is added to `allocation.unclaimed_referral_fees` in `update_referrer_on_withdraw`, and then it is subtracted from `allocation.total_referred_amount`. These values are then combined in `referral_exchange_rate` to compute the funds associated with `total_referral_supply` shares.

>_ flexlend/src/state/pool/liquidity.rs RUST

```rust
pub fn update_referrer_on_withdraw([...]) -> Result<()> {
    let (claimable_c_amount, claimable_amount) =
        referrer.calculate_claimable_referral_rewards(self)?;
    self.unclaimed_referral_fees += claimable_amount;
    referrer.referral_fee_unclaimed += claimable_amount;
    [...]
    self.total_referred_amount = self
        .total_referred_amount
        .checked_sub(withdraw_amount)
        .unwrap_or(0);
    [...]
}
```

This process involves adding `allocation.total_referred_amount` and subtracting `unclaimed_referral_fees`, resulting in an inaccurate referral exchange rate. Since `claimable_amount` is subtracted twice during each withdrawal from the total liquidity behind supply shares, it reduces rewards for referrers, potentially leaving them with none.

>_ flexlend/src/state/pool/liquidity.rs RUST

```rust
pub fn referral_exchange_rate(&self) -> Result<ExchangeRate> {
    let total_liquidity = Fraction::from(
        self.total_referred_amount + self.accumulated_referral_fees
            - self.unclaimed_referral_fees,
    );
    [...]
}
```

Remediation

Ensure that `claimable_amount` is either deducted from `total_referred_amount` or added to `unclaimed_referral_fees`, not both.

© 2024 Otter Audits LLC. All Rights Reserved.

Patch

Resolved in 5960905 and 8166c5e.

© 2024 Otter Audits LLC. All Rights Reserved.

Static Nature of Configured Weights HIGH OS-LUO-ADV-05

Description

The issue arises from the dynamic nature of the `max_exposure` calculation in `get_max_protocol_exposure` and its implications for maintaining the intended coverage as liquidity changes over time. The `max_exposure` is calculated as a weighted average of `regular_max_exposure` and `protected_max_exposure`, utilizing the respective liquidity amounts (`regular_liquidity_amount` and `protected_liquidity_amount`) as weights. At the time of calculation, `max_exposure` accurately represents Lulo's state.

>_ flexlend/src/utils/math.rs RUST

```rust
pub fn calculate_max_exposure(
    regular_tvl: u64,
    regular_max_exposure: u32,
    protected_tvl: u64,
    protected_max_exposure: u32,
) -> Result<u32> {
    [...]
    let total_balance = regular_tvl
        .checked_add(protected_tvl)
        .ok_or(error!(Errors::OverflowError))?;

    let max_exposure: u32 = total_weighted_balance
        .checked_mul(BPS_SCALE.into())
        .ok_or(error!(Errors::OverflowError))?
        .checked_div(total_balance)
        .ok_or(error!(Errors::UnderflowError))?
        .try_into()
        .map_err(|_| error!(Errors::ConversionError))?;
    Ok(max_exposure)
}
```

The protocol ensures that any newly set weights for protocol allocations (such as for Drift) are less than or equal to `max_exposure` at the time the weight is configured. However, as more funds are deposited into Lulo, the weighted average may change, causing the protocol weights to exceed the new weighted average. The protocol's risk model assumes that exposures are capped at `max_exposure`, ensuring that all potential liabilities (such as those from Drift) are covered by the liquidity pool. If Drift exceeds the new `max_exposure`, the protocol may not have sufficient coverage (via protected liquidity or reserves) to handle adverse events such as defaults or extreme market movements.

© 2024 Otter Audits LLC. All Rights Reserved.

Proof of Concept

1. The initial exposures of `LUSD` and `PUSD` are 33% and 25%, respectively.
2. As the liquidity is evenly distributed between `LUSD` and `PUSD`, `max_exposure` will be equal to 29%.
3. The `Drift`'s weight is set to 29%, which complies with the `max_exposure` at the time of calculation.
4. However, if users increase the amount of `PUSD`, `max_exposure` will decrease, approaching 25%.
5. Despite the coverage ratio limitations, the protocol continues to allocate 29% of funds to `Drift`, resulting in exposure beyond what the coverage ratio can adequately compensate.

Remediation

Periodically check and update protocol weights to reflect the latest `max_exposure` values.

Patch

Primarily resolved in b52df95, with the remaining edge case acknowledged and scheduled for future refactors.

© 2024 Otter Audits LLC. All Rights Reserved.

Risk of Outdated Allocation Data HIGH OS-LUO-ADV-06

Description

In the `initiate_regular_withdraw` instruction, `error_if_stale_refresh`, which is responsible for updating allocations with the latest rewards and ensuring their data is current, is not invoked. This may result in a stale `allocation.protected_amount` across all allocations, implying that rewards may not have been added to the variable, potentially resulting in an inaccurate exchange rate in `initiate_regular_pool_withdraw`. An outdated exchange rate may overestimate the cost of a withdrawal, burning more `PUSD` tokens than necessary, effectively penalizing the user by reducing their holdings beyond the equivalent value of the withdrawal amount.

Furthermore, `protected_amount` and `regular_amount` are utilized to determine whether a withdrawal request adheres to the pool's defined `PUSD`/`LUSD` ratio limits. The absence of a refresh implies that new rewards will not be distributed to these variables. The unaccounted rewards may impact calculations by adding more rewards to `protected_amount` than to `regular_amount`. As a result of this imbalance, the previously acceptable `PUSD/LUSD` ratio may fall out of bounds when recalculated with current values.

Remediation

Ensure that `initiate_regular_withdraw` explicitly calls `error_if_stale_refresh` at the beginning of the execution.

Patch

Resolved in c998cfe.

© 2024 Otter Audits LLC. All Rights Reserved.

Inaccuracies in Exchange Rate Calculation HIGH OS-LUO-ADV-07

Description

The issue arises due to the sequence in which state variables are updated. Specifically, the calculation of `referral_c_amount` utilizes the exchange rate derived from the state variables `total_referred_amount`, `unclaimed_referral_fees`, and `total_referral_supply`. The exchange rate calculation occurs after `unclaimed_referral_fees` has already been updated by adding the `claimable_amount`. Subsequently, the rest of the variables involved in the exchange rate calculation (`total_referred_amount` and `total_referral_supply`) are updated.

>_ *flexlend/src/state/pool/liquidity.rs* RUST

```rust
pub fn update_referrer_on_withdraw([...]) -> Result<()> {
    let (claimable_c_amount, claimable_amount) =
        referrer.calculate_claimable_referral_rewards(self)?;
    // Always add to unclaimed referral fees
    self.unclaimed_referral_fees += claimable_amount;
    referrer.referral_fee_unclaimed += claimable_amount;
    [...]
    else {
        let referral_c_amount = self
            .referral_exchange_rate()?
            .liquidity_to_collateral(withdraw_amount);
        [...]
        self.total_referred_amount = self
            .total_referred_amount
            .checked_sub(withdraw_amount)
            .unwrap_or(0);
        self.total_referral_supply = self
            .total_referral_supply
            .checked_sub(claimed_c_amount)
            .unwrap_or(0);
    }
    [...]
}
```

Thus, the state of the protocol becomes inconsistent during the calculation of exchange rate because not all variables involved in the computation reflect their new or old values simultaneously. This inconsistency arises because the exchange rate is calculated mid-state, meaning that some variables are updated before the calculation while others are updated afterward. Consequently, the exchange rate is neither fully aligned with the previous state nor fully compatible with the new state, resulting in inaccurate reward calculations.

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

Utilize the old values for the exchange rate calculation of `referral_c_amount`, and then atomically update `unclaimed_referral_fees`, `total_referred_amount`, and `total_referral_supply`.

Patch

Resolved in 144e1ce.

© 2024 Otter Audits LLC. All Rights Reserved.

Validation Failure Due to Rounding Error MEDIUM OS-LUO-ADV-08

Description

`math::calculate_accumulated_split` may fail due to rounding errors introduced when distributing rewards between the protected and regular pools. Rewards for the protected and regular pools are computed as fractions of the total `accumulated_net`, based on the liquidity contributions of each pool. The results of these divisions are rounded to the nearest whole number. This truncation may result in a scenario where the sum of `protected_rewards` and `regular_rewards` differ from the value of `accumulated_net`. Consequently, the `require` statement that checks for this condition will fail, failing the `refresh_tvl` instruction.

>_ *flexlend/src/utils/math.rs* RUST

```rust
pub fn calculate_accumulated_split(pool: &Pool, accumulated_net: u64) -> Result<(u64, u64)> {
    [...]
    require_eq!(
        protected_rewards + regular_rewards,
        accumulated_net,
        Errors::UnderflowError,
    );
    [...]
}
```

Remediation

Instead of enforcing strict equality, allow a small tolerance for rounding.

Patch

Resolved in 9c8cf5f.

© 2024 Otter Audits LLC. All Rights Reserved.

Improper Oracle Address Check MEDIUM OS-LUO-ADV-09

Description

`liquidity::update_oracle_price` checks if the `Allocation`'s stored oracle key (`self.oracle`) matches the `write_authority` of the provided `PriceUpdateV2` account (`oracle.address`). This logic is overly restrictive due to its reliance on comparing `self.oracle` to `write_authority`. This creates a vulnerability where valid updates fail unnecessarily, as this check will result in the instruction failing in cases where the `PriceUpdateV2` account's `write_authority` is anything other than itself.

>_ flexlend/src/state/pool/liquidity.rs RUST

```rust
pub fn update_oracle_price(
    [...]
) -> Result<()> {
    require_keys_eq!(self.oracle, oracle.address);
    msg!(
        "updating price for mint {}, last={}, new={}",
        self.mint,
        self.oracle_price,
        oracle.ema_price
    );
    [....]
}
```

This is especially problematic as this situation occurs in the majority of cases. As a result, valid updates to allocations will fail in cases where the oracle's `write_authority` differs from its address.

Remediation

Modify `update_oracle_price` to align with `pool::update_oracles` logic, such that the check compares the `Allocation` 's oracle address with the provided oracle's address, rather than its `write_authority`.

Patch

Resolved in a728b79.

© 2024 Otter Audits LLC. All Rights Reserved.

Utilization of Shared Pool PDA as Signer MEDIUM OS-LUO-ADV-10

Description

The issue stems from how cross-program invocations (CPI) are authenticated in the context of Lulo Flexlend's integration with supported protocols (such as MarginFi and Drift). In the current implementation, CPIs from the pool to supported protocols are signed via a single PDA (Program Derived Address). Specifically, `cpi_deposit` in Marginifi calls Marginfi with the signer as `PDA["pool", 0]`, and `cpi_deposit` in Drift does the same. This PDA acts like a global admin across all protocol interactions. This shared signature model erodes the integrity of the protocol because the compromise of one protocol equates to the compromise of all connected protocols.

Remediation

Derive protocol-specific signing authorities, where they sign with `PDA["<protocol>_authority", pool]` to ensure the required separation. Thus, for Marginfi it would be `PDA["marginfi_authority", pool]`, while for Drift it would be `PDA["drift_authority", pool]`.

Patch

Resolved in 3f5e7c9 and e1599b5.

© 2024 Otter Audits LLC. All Rights Reserved.

Incorrect Oracle Staleness Check MEDIUM OS-LUO-ADV-11

Description

`pool::error_if_stale_refresh` ensures that both the pool data and the oracle data are up to date. Here, the `elapsed_time` checks how much time has passed since the pool's last update, while `oracle_elapsed` checks how much time has passed since the oracle's last update. For the oracle check, the correct comparison should be based on how much time has passed since the oracle data was last refreshed, which is stored in `oracle_elapsed`. However, in the current code, the check incorrectly utilizes `elapsed_time`, which is derived from the pool's last update. Thus, the staleness of the oracle is checked utilizing the incorrect variable.

>_ *flexlend/src/state/pool/pool.rs* RUST

```rust
pub fn error_if_stale_refresh(&self, allocation: Allocation, current_ts: u64) -> Result<()> {
    let elapsed_time = current_ts.saturating_sub(self.last_updated);
    [...]

    let oracle_elapsed = current_ts.saturating_sub(allocation.oracle_last_updated);
    if elapsed_time > self.oracle_stale_max_seconds {
        msg!("stale oracle {}", oracle_elapsed);
        return Err(error!(Errors::OraclesNotRefreshed));
    }

    Ok(())
}
```

Remediation

Utilize `oracle_elapsed` (not `elapsed_time`) when checking the staleness of the oracle data.

Patch

Resolved in be6e66d.

© 2024 Otter Audits LLC. All Rights Reserved.

Inconsistencies in Conversion of Protocol Identifiers MEDIUM OS-LUO-ADV-12

Description

`try_into` in `Protocol` converts a `Protocol` enumeration variant into a fixed-size byte array of length eight. The vulnerability arises from the way protocol identifiers are converted into an 8-byte array. The program utilizes a fixed-length byte array of 8 bytes (`[u8; 8]`) to store protocol identifiers. This array is populated by converting protocol names (strings) into bytes and copying them into the array. When a protocol name exceeds 8 bytes, the program will always panic. This occurs because the protocol variable is defined as 8 bytes long, while `copy_from_slice` attempts to copy the first 10 bytes from `str_bytes` .

>_ *flexlend/src/protocols/mod.rs* RUST

```rust
fn try_into(self) -> Result<[u8; 8], Self::Error> {
    [...]
    let mut protocol = [0u8; 8];

    // protocol.copy_from_slice(str.as_bytes());
    // Convert input string to bytes
    let str_bytes = str.as_bytes();
    let len = str_bytes.len();
    if len > protocol.len() {
        // Truncate the input string if it's longer than `protocol.length`
        protocol.copy_from_slice(&str_bytes[..10]);
    } else {
        // Copy the input string and leave the rest as zeros if it's shorter
        protocol[..len].copy_from_slice(str_bytes);
    }
    return Ok(protocol);
}
```

Moreover, currently, all protocol identifiers have a maximum length of 8 bytes. However, if a new protocol's name exceeds this limit, the identifiers of two or more protocols may collide. This is because longer names are truncated to the first 8 bytes. As a result, the names `kamino_j` and `kamino_jup` would result in the same identifier.

Remediation

Utilize `str_bytes[..8]` , which copies only the first 8 bytes. Additionally, redesign the identifier scheme to guarantee uniqueness for all protocols.

© 2024 Otter Audits LLC. All Rights Reserved.

Patch

Resolved in 4b1a060.

© 2024 Otter Audits LLC. All Rights Reserved.

Transaction Failure Due to Drift Withdrawal Limit LOW OS-LUO-ADV-13

Description

The daily withdrawal limits enforced by `update_spot_balances_and_cumulative_deposits_with_limits` may have significant implications for the regular withdrawal process fulfilled through Drift in `complete_regular_withdraw`. In `complete_regular_withdraw`, user-initiated withdrawals are routed through the Drift protocol for execution. Drift imposes daily withdrawal limits at the spot market level, which are tracked globally and updated as withdrawals occur.

>_ flexlend/src/state/pool/liquidity.rs RUST

```rust
pub fn update_spot_balances_and_cumulative_deposits_with_limits(
   [...]
) -> DriftResult {
    [...]
    validate!(
        valid_withdraw,
        ErrorCode::DailyWithdrawLimit,
        "Spot Market {} has hit daily withdraw limit. Attempted withdraw amount of {} by {}",
        spot_market.market_index,
        token_amount,
        user.authority
    )?;
    [...]
}
```

`update_spot_balances_and_cumulative_deposits_with_limits` verifies whether a withdrawal remains within Drift's daily withdrawal limit for a specific `spot_market`. If the withdrawal amount exceeds the allowed daily cap, the operation is rejected. Thus, if a market has already reached its daily withdrawal cap (due to high withdrawal activity or whale transactions), subsequent user withdrawal requests for that market will fail.

Remediation

Check Drift's remaining withdrawal capacity for the requested spot market and inform users proactively if the market has hit its limit, avoiding wasted attempts.

Patch

Acknowledged by Lulo team.

© 2024 Otter Audits LLC. All Rights Reserved.

Unfavorable Rounding Direction LOW OS-LUO-ADV-14

Description

The `PUSD` / `LUSD` amount, which is calculated in the withdrawal instructions (`withdraw_protected_pool`), is currently rounded down in `liquidity::liquidity_to_collateral`. If the collateral amount is rounded down, it reduces the number of `c-tokens` burned when a user withdraws. As a result, the user withdraws slightly more value from the pool without burning an equivalent amount of their c-tokens.

>_ flexlend/src/state/pool/liquidity.rs RUST

```rust
pub fn liquidity_to_collateral(&self, liquidity_amount: u64) -> u64 {
    (self.0 * u128::from(liquidity_amount)).to_floor()
}
```

When repeated across multiple withdrawals, the rounding-down mechanism will create a cumulative discrepancy in the pool. This diminishes the collateral available in the pool as it absorbs the cost of these discrepancies. Thus, the current rounding strategy favors the withdrawing users at the expense of the pool and remaining liquidity providers.

Remediation

Modify the rounding mechanism such that the collateral amount is rounded up. This will favor the pool, as it ensures that the user burns slightly more `c-tokens` than the exact share required for their withdrawal.

Patch

Resolved in 5d4a289.

© 2024 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-LUO-SUG-00	The reliance on Solana's `Clock::get` for time-based price validation may introduce inaccuracies due to potential clock drift.
OS-LUO-SUG-01	Allowing multiple user-provided accounts for `pool_input_token_account` results in the fragmentation of funds across different accounts, increasing the complexity of tracking and transferring funds back to the original pool account.
OS-LUO-SUG-02	`deposit_pool` does not verify `user_account` ownership, which is mitigated by runtime error prevention when fields are modified. However, ownership checks are still recommended to enhance security.
OS-LUO-SUG-03	There are several instances where proper validation is not performed, resulting in potential security issues.
OS-LUO-SUG-04	Suggestions to ensure adherence to coding best practices.
OS-LUO-SUG-05	Recommendations for modifying the codebase for improved functionality, efficiency, and maintainability.
OS-LUO-SUG-06	The codebase contains multiple cases of redundant code that should be removed for better maintainability and clarity.

© 2024 Otter Audits LLC. All Rights Reserved.

Lack of Price Freshness Validation OS-LUO-SUG-00

Description

Solana's `Clock::get` provides the blockchain's perceived current time. However, this time is not always guaranteed to be precise and may drift and become inaccurate, affecting the Pyth price calculations, as inaccurate timing may result in accepting stale prices. To address this issue, Pyth's team utilizes a `posted_slot` as an additional proxy measure for recency in the program. While `publish_time` is already checked within `price::price_is_not_stale`, `posted_slot` in `price_update` is not currently checked. Ideally, if either of `posted_slot` or `publish_time` is outdated, the oracle should be considered stale.

>_ price_update.rs RUST

```rust
#[account]
#[derive(BorshSchema)]
pub struct PriceUpdateV2 {
    pub write_authority: Pubkey,
    pub verification_level: VerificationLevel,
    pub price_message: PriceFeedMessage,
    pub posted_slot: u64,
}
```

Remediation

Add a check to ensure if `posted_slot` in `price_update` is outdated, the oracle data is considered stale.

© 2024 Otter Audits LLC. All Rights Reserved.

Token Account Fragmentation OS-LUO-SUG-01

Description

In the current implementation, users may provide multiple different accounts in place of `pool_input_token_account` owned by the pool. While this does not pose a security risk, allowing multiple types of accounts to hold these tokens introduces fragmentation of funds, scattering them into multiple accounts. Consequently, the application will have to search through multiple accounts to find all funds that should be transferred to the original `pool_input_token_account`, affecting the efficiency of the protocol.

>_ *flexlend/src/instructions/pool/complete_regular_withdraw.rs* RUST

```rust
#[derive(Accounts)]
#[instruction(withdrawal_id: u16)]
pub struct CompleteWithdraw<'info> {
    [...]
    #[account(
        init_if_needed,
        payer = fee_payer,
        associated_token::mint = input_mint,
        associated_token::authority = pool,
        associated_token::token_program = input_mint_token_program,
    )]
    pub pool_input_token_account: InterfaceAccount<'info, TokenAccount>,
    [...]
}
```

Remediation

Enforce that `pool_input_token_account` must be an associated token account for the pool.

© 2024 Otter Audits LLC. All Rights Reserved.

Absence of Ownership Verification

OS-LUO-SUG-02

Description

The `deposit_pool` instruction in Lulo accepts an `UncheckedAccount` as the `user_account` parameter and deserializes it into a `UserAccount`. However, ownership of the account is not verified. Currently, it is not exploitable, as `deposit_pool` modifies specific fields in `user_account`, and Solana's runtime enforces that only the owner program may modify its account data—attempts to modify an account not owned by the Lulo program will result in a runtime error, failing the instruction. Nonetheless, explicitly verifying ownership would provide additional assurance and reduce reliance on implicit runtime checks.

>_ *flexlend/src/instructions/pool/deposit_pool.rs* RUST

```rust
#[derive(Accounts)]
pub struct DepositPool<'info> {
    [...]
    /// CHECK: user_account can be initialized in the instruction
    #[account(mut)]
    pub user_account: UncheckedAccount<'info>,
    [...]
}
```

Remediation

Explicitly check that the `user_account`'s PDA (Program-Derived Address) is controlled by the Lulo program.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-LUO-SUG-03

Description

1. The check in `liquidity::add_pending_withdraw` verifies whether the current withdrawal amount is less than the available balance in the protocol (`deposited_drift`). However, this check only validates the individual withdrawal and does not consider the total sum of all pending withdrawals. The logic, as it stands, does not ensure that the combined total of ongoing (already pending) withdrawals plus the new one will not exceed the total available funds in the protocol.

   >_ flexlend/src/state/pool/liquidity.rs RUST

   ```rust
   pub fn add_pending_withdraw(&mut self, amount: u64) -> Result<u64> {
       let deposited_drift = self.get_protocol(Protocol::Drift)?.balance_native;
       if amount > deposited_drift || amount > deposited_drift {
           msg!(
               "cant initiate withdrawal for {}, balance={}, total_liquidity={}",
               amount,
               deposited_drift,
               self.total_liquidity,
           );
           return Err(error!(Errors::InvalidPoolWithdraw));
       }
       [...]
   }
   ```

2. In the configuration update logic for `pool/admin_parameters`, only `update_max_exposures` includes explicit sanitization of input parameters through `check_exposure`. Other administrative configuration update functions lack similar validation mechanisms. `update_oracle_stale_maximum` , `update_refresh_stale_maximum` , `update_withdraw_cooldown` , and others directly update the pool state without checking whether the provided parameters are valid within a defined context.

Remediation

1. Verify that the total of `pending_withdrawals` (the sum of all current pending withdrawals) and the new withdrawal `amount` does not exceed the available funds.
2. Introduce parameter validation for every update function to ensure proper input sanitization.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-LUO-SUG-04

Description

1. Currently, in `add_allocation` instruction, the decimals for `Allocation`'s `input_mint` must always have a decimal precision of 8 for the code to function correctly. However, this assumption has not been explicitly checked anywhere.
2. In `pool::initialize`, there is a discrepancy between the comment and the actual value assigned to `referral_fee_bps`. The code sets the `referral_fee_bps` value to 50%, while the comment incorrectly states it as 10%. Ensure consistency between the comments and the actual implementation.

>_ flexlend/src/state/pool/pool.rs RUST

```rust
pub fn initialize(
    [...]
) -> Result<&mut Pool> {
    [...]
    self.referral_fee_bps = decimal_to_bps(0.50).try_into().unwrap(); // 10% protocol fee
    self.protected_interest_share_bps = decimal_to_bps(0.50).try_into().unwrap(); // 50%
        ↪  protected interest share
    self.coverage_float_bps = decimal_to_bps(0.90).try_into().unwrap();
    Ok(self)
}
```

3. The current design of Lulo mints `LUSD` (regular mint) and `PUSD` (protected mint) tokens into accounts under the protocol's custody, but it is not possible to transfer these to other users. This restriction limits the utility of these tokens as fungible assets, essentially treating them as non-transferable accounting records. Instead of minting tokens, the user's balance of `LUSD` and `PUSD` may be tracked as numerical fields, updated during deposits, withdrawals, and other operations, achieving the same functionality while simplifying the implementation.

Remediation

Implement the above-mentioned suggestions.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-LUO-SUG-05

Description

1. The sanity check on `amount` in the `withdraw_protected_pool` instruction may be enhanced by adding a minimum balance requirement for the token account. This will ensure that the withdrawal amount exceeds the sum of the pool amount and the minimum token account balance, confirming the pool has sufficient liquidity to cover the requested withdrawal.

>_ flexlend/src/instructions/pool /withdraw_protected_pool.rs RUST

```rust
pub fn withdraw_protected_pool<'a, 'b, 'c, 'info>(
    ctx: Context<'a, 'b, 'c, 'info, WithdrawProtectedPool<'info>>,
    amount: u64,
    withdraw_all: bool,
) -> Result<()> {
    [...]
    if amount > pool_token_amount {
        return Err(error!(Errors::NotEnoughtInputPoolBalance));
    }
    [...]
}
```

2. In the existing implementation of `liquidity::add_pending_withdraw`, the check to ascertain if the `amount` is greater than `deposited_drift` is performed twice unnecessarily. Update the `if` condition to remove the redunant check.

>_ flexlend/src/state/pool/liquidity.rs RUST

```rust
pub fn add_pending_withdraw(&mut self, amount: u64) -> Result<u64> {
    [...]
    if amount > deposited_drift || amount > deposited_drift {
        msg!([...]);
        return Err(error!(Errors::InvalidPoolWithdraw));
    }
    [...]
}
```

3. A user may still withdraw all their tokens in `initiate_regular_pool_withdraw` even if they did not provide a `withdraw_all` flag. To mitigate this, the `c_amount` should be compared to the user's total balance (`user_lp_token_amount`), as done in `withdraw_protected_pool` . If they match, the `withdraw_all` flag should be implicitly set to true, indicating the user is attempting to withdraw all their tokens.

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

Incorporate the above-stated refactors.

© 2024 Otter Audits LLC. All Rights Reserved.

Redundant Code

OS-LUO-SUG-06

Description

1. In `initiate_regular_withdraw`, `owner_input_token_account` and `pool_input_token_account` are redundant.
2. In `complete_regular_withdraw`, `flex_use_lp_token_account` is redundant.

Remediation

Remove the redundant code instances highlighted above.

© 2024 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2024 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2024 Otter Audits LLC. All Rights Reserved.